# C++ Object-Oriented Programming - Complete Interview Guide

## Theory, Edge Cases & Code Examples

- ✓ Encapsulation, Inheritance, Polymorphism
- ✓ The Rule of Three/Five/Zero
- ✓ Common Pitfalls
- ✓ Virtual Functions in Constructors and Destructors
- ✓ Multiple Definitions of Special Members

→ *Swipe to learn* →

# C++ Object-Oriented Programming - Complete Interview Guide

## Theory

## 📌 1. The Three Pillars of C++ OOP - Encapsulation, Inheritance, Polymorphism (1/3)

### Core OOP principles:

| Pillar | Definition | C++ Mechanism | Benefit |
|---|---|---|---|
| **Encapsulation** | Bundling data and methods together with access control | `private`, `protected`, `public` keywords | Data hiding, invariant enforcement, controlled interfaces |
| **Inheritance** | Deriving new classes from existing ones | `: public Base`, `: protected Base`, `: private Base` | Code reuse, hierarchical modeling, "is-a" relationships |
| **Polymorphism** | One interface, multiple implementations | Virtual functions, function/operator overloading, templates | Flexibility, extensibility, runtime behavior selection |

### Class vs Struct - only ONE difference:

| Aspect | `struct` | `class` |
|---|---|---|
| **Default member access** | `public` | `private` |
| **Default inheritance mode** | `public` | `private` |
| **All other features** | ✅ **Identical** (constructors, destructors, inheritance, virtual functions, etc.) | ✅ **Identical** |
| **Convention** | Simple data aggregates (POD-like) | Objects with behavior and encapsulation |
| **Memory layout** | **Exactly the same** | **Exactly the same** |

# C++ Object-Oriented Programming - Complete Interview Guide

## Theory

## 📌 1. The Three Pillars of C++ OOP - Encapsulation, Inheritance, Polymorphism (2/3)

### Compile-time vs Runtime polymorphism:

| Type | Mechanism | Resolution Time | Examples | Performance |
|---|---|---|---|---|
| **Compile-time (Static)** | Function/operator overloading, templates | Compile time | `void func(int)` vs `void func(double)`, `template<T> func(T)` | ✅ Zero overhead |
| **Runtime (Dynamic)** | Virtual functions via vtable | Runtime | `virtual void func()` overridden in derived classes | Small vtable lookup cost |

# C++ Object-Oriented Programming - Complete Interview Guide

## Theory

## 📌 1. The Three Pillars of C++ OOP - Encapsulation, Inheritance, Polymorphism (3/3)

### Code example - runtime polymorphism:

```
class Shape {
public:
    virtual double area() const = 0;  // Runtime polymorphism
    virtual ~Shape() = default;
};

class Circle : public Shape {
    double radius;
public:
    Circle(double r) : radius(r) {}
    double area() const override {
        return 3.14159 * radius * radius;
    }
};

class Rectangle : public Shape {
    double width, height;
public:
    Rectangle(double w, double h) : width(w), height(h) {}
    double area() const override {
        return width * height;
    }
};

// ✅ Polymorphic function - works with any Shape
void printArea(const Shape& shape) {
    std::cout << "Area: " << shape.area() << "\n";  // Runtime dispatch
}
```

# C++ Object-Oriented Programming - Complete Interview Guide

## Theory

## 📌 2. Special Member Functions - The Rule of Three/Five/Zero (1/4)

### The six special member functions:

| Function | Signature | When Auto-Generated | Purpose |
|---|---|---|---|
| **Default Constructor** | `T()` | If no other constructors defined | Initialize object with default values |
| **Destructor** | `~T()` | Always (if not user-defined) | Clean up resources |
| **Copy Constructor** | `T(const T&)` | If no move operations declared | Create new object as copy |
| **Copy Assignment** | `T& operator=(const T&)` | If no move operations declared | Assign to existing object |
| **Move Constructor** | `T(T&&)` | If no special members user-declared | Transfer resources from temporary |
| **Move Assignment** | `T& operator=(T&&)` | If no special members user-declared | Transfer resources to existing object |

### The three rules compared:

| Rule | Era | When to Apply | What to Define | Modern Preference |
|---|---|---|---|---|
| **Rule of Zero** | C++11+ | Class doesn't manage resources | **Nothing** - use `unique_ptr`, `vector`, `string` | ✅ **Strongly preferred** |
| **Rule of Three** | C++98 | Class manages resources (pre-move) | Destructor + Copy Ctor + Copy Assign | Legacy code only |
| **Rule of Five** | C++11+ | Class manages resources (with move) | Rule of Three + Move Ctor + Move Assign | When Rule of Zero impossible |

# C++ Object-Oriented Programming - Complete Interview Guide

## Theory

## 📌 2. Special Member Functions - The Rule of Three/Five/Zero (2/4)

### Decision tree - which rule to follow:

```
Does your class directly own resources (raw pointers, file handles, etc.)?
│
├─ NO  → ✅ Follow Rule of Zero
│         Use smart pointers and RAII wrappers
│         Let compiler generate everything
│         Example: std::unique_ptr<int[]>, std::vector<T>
│
└─ YES → Follow Rule of Five
          Define/delete all five special member functions
          Ensure move operations are noexcept
          Example: Custom allocators, legacy C interface wrappers
```

### Code example - Rule of Zero (MODERN APPROACH):

```
class ModernBuffer {
    std::unique_ptr<int[]> data;  // ✅ Smart pointer manages memory
    size_t size;
public:
    explicit ModernBuffer(size_t n)
        : data(std::make_unique<int[]>(n)), size(n) {}

    // ✅ No special member functions defined
    // unique_ptr provides correct move-only semantics automatically
    // Compiler generates: destructor, move ctor, move assign
    // Copy is deleted (unique_ptr is move-only)
};
```

# C++ Object-Oriented Programming - Complete Interview Guide

## Theory

## 📌 2. Special Member Functions - The Rule of Three/Five/Zero (3/4)

### Code example - Rule of Five (when necessary):

```
class LegacyBuffer {
    int* data;  // Raw pointer (C interface, legacy code)
    size_t size;
public:
    explicit LegacyBuffer(size_t n) : size(n), data(new int[n]) {}

    // ✅ Rule of Five - all five explicitly defined
    ~LegacyBuffer() { delete[] data; }

    LegacyBuffer(const LegacyBuffer& other)
        : size(other.size), data(new int[other.size]) {
        std::copy(other.data, other.data + size, data);
    }

    LegacyBuffer& operator=(const LegacyBuffer& other) {
        if (this != &other) {
            int* new_data = new int[other.size];
            std::copy(other.data, other.data + other.size, new_data);
            delete[] data;
            data = new_data;
            size = other.size;
        }
        return *this;
    }

    LegacyBuffer(LegacyBuffer&& other) noexcept
        : data(other.data), size(other.size) {
        other.data = nullptr;
        other.size = 0;
    }

    LegacyBuffer& operator=(LegacyBuffer&& other) noexcept {
        if (this != &other) {
            delete[] data;
            data = other.data;
            size = other.size;
            other.data = nullptr;
            other.size = 0;
        }
// ...continues on next slide
```

## 📌 2. Special Member Functions - The Rule of Three/Five/Zero (4/4)

[Code continues across slides]

```
// ...continued from previous slide
        return *this;
    }
};
```

# C++ Object-Oriented Programming - Complete Interview Guide

## Theory

## 📌 3. Resource Management Dangers - Common Pitfalls (1/2)

**What happens with wrong resource management:**

| Mistake | Symptom | Cause | Fix |
|---|---|---|---|
| **Double-deletion crash** | Crash when second object destroyed | Shallow copy with raw pointers | Deep copy or Rule of Zero |
| **Memory leak** | Growing memory usage | No destructor or improper cleanup | RAII and proper destructor |
| **Use-after-free** | Crash or undefined behavior | Dangling pointer after move | Nullify pointers in move operations |
| **Resource leak** | File handles/sockets exhausted | Missing cleanup in exception paths | RAII wrappers auto-cleanup |
| **Object slicing** | Lost data and polymorphism | Pass polymorphic types by value | Pass by reference/pointer |

**Why Rule of Zero is strongly preferred:**

| Rule of Zero Benefit | Why It Matters |
|---|---|
| **No manual memory management** | Eliminates entire class of bugs (leaks, double-delete) |
| **Exception-safe by default** | RAII wrappers clean up even during exceptions |
| **Move semantics automatic** | unique_ptr, vector, etc. provide optimal moves |
| **Less code to maintain** | Compiler-generated special members always correct |
| **Clearer intent** | Composition shows exactly what resources are owned |
| **Standard library tested** | Battle-tested implementations vs custom code |

# C++ Object-Oriented Programming - Complete Interview Guide

## Theory

## 📌 3. Resource Management Dangers - Common Pitfalls (2/2)

### Code example - dangers of shallow copy:

```
class DangerousBuffer {
    int* data;
    size_t size;
public:
    DangerousBuffer(size_t n) : size(n), data(new int[n]) {}
    ~DangerousBuffer() { delete[] data; }

    // ❌ Compiler-generated copy constructor does shallow copy!
    // Both objects point to same memory
};

int main() {
    DangerousBuffer buf1(100);
    DangerousBuffer buf2 = buf1;  // ❌ Shallow copy
    // When buf2 destroys: delete[] data
    // When buf1 destroys: delete[] data again (same pointer!)
    // CRASH: Double-deletion
}
```

**Key takeaway:** Follow Rule of Zero using smart pointers and RAII wrappers whenever possible. Only use Rule of Five when directly managing resources for legacy interfaces or custom allocators. Understanding OOP pillars (encapsulation, inheritance, polymorphism) combined with proper resource management is essential for safe, efficient C++ programming.

# C++ Object-Oriented Programming - Complete Interview Guide

## Edge Cases

### 📌 1. Virtual Functions in Constructors and Destructors

Calling virtual functions from constructors or destructors doesn't work as expected due to how objects are constructed. During base class construction, the derived portion doesn't exist yet, so the vptr points to the base class vtable. Calling a pure virtual function from a constructor causes undefined behavior, often crashing.

```
class Base {
public:
    Base() {
        init();  // ❌ Dangerous: calls pure virtual during construction
    }
    virtual void init() = 0;
};

class Derived : public Base {
public:
    void init() override {
        std::cout << "Derived init\n";
    }
};

int main() {
    Derived d;  // ❌ Undefined behavior or crash
}
```

During Base construction, the object's type is Base, not Derived. The vptr points to Base's vtable which has an invalid entry for the pure virtual init(). Attempting to call it accesses an invalid pointer. Never call virtual functions from constructors or destructors—use two-phase initialization instead.

# C++ Object-Oriented Programming - Complete Interview Guide

## Edge Cases

### 📌 2. Multiple Definitions of Special Members

When you define some special members but not others, the compiler's generation rules create subtle interdependencies. Defining a destructor suppresses automatic move operation generation. Defining a move constructor suppresses copy operation generation. These rules changed between C++ standards, creating version-dependent behavior.

```
class Problem {
public:
    ~Problem() {}  // ❌ User-defined destructor
    // Move constructor/assignment NOT generated
    // Copy operations still generated (deprecated)
};

static_assert(!std::is_move_constructible_v<Problem>);
// Problem is not move-constructible despite appearing default

class Fixed {
public:
    ~Fixed() {}
    Fixed(Fixed&&) = default;  // ✅ Explicitly default move
    Fixed& operator=(Fixed&&) = default;
    Fixed(const Fixed&) = default;  // ✅ Explicitly default copy
    Fixed& operator=(const Fixed&) = default;
};
```

The automatic generation rules are complex and changed over C++ versions for backward compatibility. Following Rule of Five by explicitly defaulting or deleting all five special members eliminates ambiguity and makes intent clear.

## 📌 3. Private Copy with Public Move

An unusual but valid pattern is making a class copyable only internally (private copy operations) while publicly movable. This enforces single ownership while allowing controlled internal duplication.

```
class SemiUnique {
private:
    std::unique_ptr<int> data;

    SemiUnique(const SemiUnique& other)  // ✅ Private copy
        : data(std::make_unique<int>(*other.data)) {}

public:
    SemiUnique(int value) : data(std::make_unique<int>(value)) {}

    SemiUnique(SemiUnique&&) = default;  // ✅ Public move
    SemiUnique& operator=(SemiUnique&&) = default;

    friend SemiUnique duplicate(const SemiUnique& obj) {
        return obj;  // ✅ Friend can use private copy
    }
};
```

This pattern is useful for types that should generally transfer ownership but need controlled duplication through specific APIs. It prevents accidental copying while preserving move semantics.

# C++ Object-Oriented Programming - Complete Interview Guide

## Edge Cases

### 📌 4. Derived Class Cannot Override Non-Virtual Destructor

Even if a derived class defines its own destructor, without a virtual base destructor, the derived destructor won't be called when deleting through a base pointer. This is a critical source of resource leaks.

```
class Base {
public:
    ~Base() { std::cout << "~Base\n"; }  // ❌ Not virtual
};

class Derived : public Base {
    int* largeArray;
public:
    Derived() : largeArray(new int[10000]) {}
    ~Derived() {
        delete[] largeArray;  // ❌ Never called through base pointer
        std::cout << "~Derived\n";
    }
};

int main() {
    Base* ptr = new Derived();
    delete ptr;  // Only prints "~Base", leaks largeArray
}
```

The fix is simple: make Base's destructor virtual. This enables proper polymorphic destruction through the vtable mechanism. Many coding standards require virtual destructors for all polymorphic base classes.

# C++ Object-Oriented Programming - Complete Interview Guide

## Edge Cases

### 📌 5. Implicit Conversions and Explicit Constructors

Single-parameter constructors act as implicit conversion functions unless marked `explicit`. This can cause surprising behavior in overload resolution and function calls.

```
class String {
public:
    String(const char* str);  // ❌ Allows implicit conversion
};

void process(String s);

process("hello");  // Implicitly converts const char* to String

class SafeString {
public:
    explicit SafeString(const char* str);  // ✅ Requires explicit construction
};

void process2(SafeString s);

// process2("hello");  // ❌ Error: no implicit conversion
process2(SafeString("hello"));  // ✅ OK: explicit construction
```

Mark single-parameter constructors `explicit` unless implicit conversion is genuinely desired and makes semantic sense. Copy and move constructors are exempt from this guideline as their implicit behavior is expected.

# C++ Object-Oriented Programming - Complete Interview Guide

## Edge Cases

### 📌 6. Name Hiding in Inheritance

When a derived class defines a function with the same name as a base function (but different signature), it hides all base overloads of that name. This is function hiding, not overloading, and can cause confusing behavior.

```
class Base {
public:
    virtual void func(int x) { std::cout << "Base::func(int)\n"; }
    virtual void func(double x) { std::cout << "Base::func(double)\n"; }
};

class Derived : public Base {
public:
    void func(int x) override { std::cout << "Derived::func(int)\n"; }
    // Base::func(double) is hidden, not overridden
};

int main() {
    Derived d;
    d.func(42);     // ✅ Calls Derived::func(int)
    // d.func(3.14);  // ❌ Error: no matching function

    Base* b = &d;
    b->func(3.14);  // ✅ Calls Base::func(double) via vtable
}
```

To bring hidden base functions into scope, use a `using` declaration: `using Base::func;`. This restores overload resolution while allowing selective overriding.

# C++ Object-Oriented Programming - Complete Interview Guide

## Code Examples

### 📌 1. Complete Rule of Five Implementation (1/2)

This demonstrates complete Rule of Five implementation with proper resource management, including deep copying for copy operations, ownership transfer for move operations, and self-assignment checks.

```
class ResourceManager {
    int* data;
    size_t size;

public:
    // Constructor
    ResourceManager(size_t n) : size(n), data(new int[n]) {
        std::cout << "Constructed with size " << size << "\n";
    }

    // ✅ Copy constructor - deep copy
    ResourceManager(const ResourceManager& other)
        : size(other.size), data(new int[other.size]) {
        std::copy(other.data, other.data + size, data);
        std::cout << "Copy constructed\n";
    }

    // ✅ Copy assignment - deep copy with self-assignment check
    ResourceManager& operator=(const ResourceManager& other) {
        if (this != &other) {
            delete[] data;
            size = other.size;
            data = new int[size];
            std::copy(other.data, other.data + size, data);
            std::cout << "Copy assigned\n";
        }
        return *this;
    }

    // ✅ Move constructor - transfer ownership
    ResourceManager(ResourceManager&& other) noexcept
        : size(other.size), data(other.data) {
        other.data = nullptr;
        other.size = 0;
        std::cout << "Move constructed\n";
    }
// ...continues on next slide
```

# C++ Object-Oriented Programming - Complete Interview Guide

## Code Examples

### 📌 1. Complete Rule of Five Implementation (2/2)

[Code continues across slides]

```
// ...continued from previous slide

    // ✅ Move assignment - transfer ownership
    ResourceManager& operator=(ResourceManager&& other) noexcept {
        if (this != &other) {
            delete[] data;
            data = other.data;
            size = other.size;
            other.data = nullptr;
            other.size = 0;
            std::cout << "Move assigned\n";
        }
        return *this;
    }

    // ✅ Destructor
    ~ResourceManager() {
        delete[] data;
        std::cout << "Destroyed\n";
    }
};
```

# C++ Object-Oriented Programming - Complete Interview Guide

## Code Examples

### 📌 2. Polymorphic Interface with Virtual Destructor (1/2)

This shows a clean polymorphic interface with virtual destructor, protected construction to prevent slicing, and deleted copy operations since loggers represent unique resources.

```
class ILogger {
public:
    virtual void log(const std::string& message) = 0;
    virtual ~ILogger() = default;  // ✅ Virtual destructor

protected:
    ILogger() = default;
    ILogger(const ILogger&) = delete;  // ✅ Prevent slicing
    ILogger& operator=(const ILogger&) = delete;
};

class FileLogger : public ILogger {
    std::ofstream file;
public:
    explicit FileLogger(const std::string& filename) {
        file.open(filename, std::ios::app);
    }

    void log(const std::string& message) override {
        if (file.is_open()) {
            file << message << "\n";
        }
    }

    ~FileLogger() override {
        if (file.is_open()) {
            file.close();
        }
    }
};

class ConsoleLogger : public ILogger {
public:
    void log(const std::string& message) override {
        std::cout << "[LOG] " << message << "\n";
    }
// ...continues on next slide
```

## 📌 2. Polymorphic Interface with Virtual Destructor (2/2)

[Code continues across slides]

```
// ...continued from previous slide
};

void processWithLogger(ILogger& logger) {
    logger.log("Processing started");
    // ... processing logic
    logger.log("Processing completed");
}
```

# C++ Object-Oriented Programming - Complete Interview Guide

## Code Examples

### 📌 3. Move-Only Type Pattern (1/2)

Move-only types represent unique ownership where duplication doesn't make sense. Deleting copy operations and providing moves enables efficient transfer while preventing accidental duplication.

```
class FileHandle {
    FILE* handle;
    std::string filename;

public:
    explicit FileHandle(const std::string& fname) : filename(fname) {
        handle = fopen(fname.c_str(), "r");
        if (!handle) {
            throw std::runtime_error("Cannot open file: " + fname);
        }
    }

    // ❌ Delete copy operations - file handles are unique
    FileHandle(const FileHandle&) = delete;
    FileHandle& operator=(const FileHandle&) = delete;

    // ✅ Move operations - transfer ownership
    FileHandle(FileHandle&& other) noexcept
        : handle(other.handle), filename(std::move(other.filename)) {
        other.handle = nullptr;
    }

    FileHandle& operator=(FileHandle&& other) noexcept {
        if (this != &other) {
            if (handle) fclose(handle);
            handle = other.handle;
            filename = std::move(other.filename);
            other.handle = nullptr;
        }
        return *this;
    }

    ~FileHandle() {
        if (handle) {
            fclose(handle);
        }
// ...continues on next slide
```

## 📌 3. Move-Only Type Pattern (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    }

    bool isOpen() const { return handle != nullptr; }
};
```

# C++ Object-Oriented Programming - Complete Interview Guide

## Code Examples

### 📌 4. Two-Phase Initialization Pattern (1/2)

Two-phase initialization separates construction from virtual initialization, avoiding the problem of calling virtual functions from constructors.

```
class DatabaseConnection {
    std::string connectionString;
    bool connected;

protected:
    // ✅ Constructor doesn't call virtual functions
    DatabaseConnection(std::string connStr)
        : connectionString(connStr), connected(false) {
        // No virtual calls here
    }

    // ✅ Virtual initialization function
    virtual bool connectImpl() = 0;

public:
    bool connect() {
        if (!connected) {
            connected = connectImpl();
        }
        return connected;
    }

    bool isConnected() const { return connected; }

    virtual ~DatabaseConnection() = default;
};

class MySQLConnection : public DatabaseConnection {
public:
    MySQLConnection(std::string connStr)
        : DatabaseConnection(connStr) {}

protected:
    bool connectImpl() override {
        std::cout << "Connecting to MySQL...\n";
        // Actual MySQL connection logic
// ...continues on next slide
```

## 📌 4. Two-Phase Initialization Pattern (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        return true;
    }
};

int main() {
    MySQLConnection conn("localhost:3306");
    conn.connect();  // ✅ Safe: called after construction
}
```

# C++ Object-Oriented Programming - Complete Interview Guide

## Code Examples

### 📌 5. Virtual Clone Pattern (1/2)

The virtual clone pattern provides polymorphic copying for class hierarchies where copy constructors would cause slicing. Each derived class knows how to clone itself correctly.

```
class Shape {
public:
    virtual std::unique_ptr<Shape> clone() const = 0;
    virtual void draw() const = 0;
    virtual ~Shape() = default;

protected:
    Shape() = default;
    Shape(const Shape&) = default;
};

class Circle : public Shape {
    double radius;
public:
    explicit Circle(double r) : radius(r) {}

    std::unique_ptr<Shape> clone() const override {
        return std::make_unique<Circle>(*this);
    }

    void draw() const override {
        std::cout << "Drawing circle with radius " << radius << "\n";
    }
};

class Rectangle : public Shape {
    double width, height;
public:
    Rectangle(double w, double h) : width(w), height(h) {}

    std::unique_ptr<Shape> clone() const override {
        return std::make_unique<Rectangle>(*this);
    }

    void draw() const override {
        std::cout << "Drawing rectangle " << width << "x" << height << "\n";
// ...continues on next slide
```

## 📌 5. Virtual Clone Pattern (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    }
};

std::vector<std::unique_ptr<Shape>> cloneShapes(
    const std::vector<std::unique_ptr<Shape>>& shapes) {
    std::vector<std::unique_ptr<Shape>> clones;
    for (const auto& shape : shapes) {
        clones.push_back(shape->clone());  // ✅ Polymorphic copying
    }
    return clones;
}
```

# C++ Object-Oriented Programming - Complete Interview Guide

## Code Examples

### 📌 6. Private Destructor Singleton Pattern (1/2)

The Singleton pattern uses private destructor (or deleted in modern C++) and deleted copy/move operations to ensure exactly one instance exists and is never duplicated.

```
class ConfigManager {
private:
    static ConfigManager* instance;
    std::map<std::string, std::string> config;

    // ✅ Private constructor and destructor
    ConfigManager() {
        std::cout << "ConfigManager created\n";
    }

    ~ConfigManager() {
        std::cout << "ConfigManager destroyed\n";
    }

    // ❌ Delete copy and move
    ConfigManager(const ConfigManager&) = delete;
    ConfigManager& operator=(const ConfigManager&) = delete;
    ConfigManager(ConfigManager&&) = delete;
    ConfigManager& operator=(ConfigManager&&) = delete;

public:
    static ConfigManager& getInstance() {
        static ConfigManager instance;
        return instance;
    }

    void set(const std::string& key, const std::string& value) {
        config[key] = value;
    }

    std::string get(const std::string& key) const {
        auto it = config.find(key);
        return it != config.end() ? it->second : "";
    }
};

// ...continues on next slide
```

## 📌 6. Private Destructor Singleton Pattern (2/2)

[Code continues across slides]

```
// ...continued from previous slide
// ConfigManager* ConfigManager::instance = nullptr;

int main() {
    ConfigManager& config = ConfigManager::getInstance();
    config.set("app_name", "MyApp");
    std::cout << config.get("app_name") << "\n";

    // ConfigManager c = config;  // ❌ Error: deleted
}
```

# C++ Object-Oriented Programming - Complete Interview Guide

## Code Examples

## 📌 7. Abstract Factory with Virtual Constructor Pattern (1/2)

The factory pattern provides a "virtual constructor" mechanism through static factory methods, enabling object creation without exposing concrete classes.

```
class Document {
public:
    virtual void open() = 0;
    virtual void save() = 0;
    virtual void close() = 0;
    virtual ~Document() = default;
};

class PDFDocument : public Document {
public:
    void open() override { std::cout << "Opening PDF\n"; }
    void save() override { std::cout << "Saving PDF\n"; }
    void close() override { std::cout << "Closing PDF\n"; }
};

class WordDocument : public Document {
public:
    void open() override { std::cout << "Opening Word doc\n"; }
    void save() override { std::cout << "Saving Word doc\n"; }
    void close() override { std::cout << "Closing Word doc\n"; }
};

class DocumentFactory {
public:
    enum class Type { PDF, WORD };

    // ✅ Virtual constructor pattern via factory method
    static std::unique_ptr<Document> create(Type type) {
        switch (type) {
            case Type::PDF:
                return std::make_unique<PDFDocument>();
            case Type::WORD:
                return std::make_unique<WordDocument>();
            default:
                return nullptr;
        }
// ...continues on next slide
```

## 📌 7. Abstract Factory with Virtual Constructor Pattern (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    }

    virtual ~DocumentFactory() = default;
};

int main() {
    auto doc = DocumentFactory::create(DocumentFactory::Type::PDF);
    doc->open();
    doc->save();
    doc->close();
}
```

# C++ Object-Oriented Programming - Complete Interview Guide

## Code Examples

## 📌 8. RAII and Rule of Zero

Following Rule of Zero with RAII types eliminates manual resource management. The compiler-generated special members correctly handle ownership based on member types.

```
class ModernResourceManager {
    std::unique_ptr<int[]> data;  // ✅ Smart pointer manages memory
    std::vector<std::string> strings;  // ✅ Vector manages elements
    size_t capacity;

public:
    explicit ModernResourceManager(size_t n)
        : data(std::make_unique<int[]>(n)), capacity(n) {
        strings.reserve(n);
    }

    // ✅ No special member functions needed
    // Compiler generates correct copy/move/destructor
    // based on members (unique_ptr is move-only, so class is move-only)

    void addString(const std::string& str) {
        strings.push_back(str);
    }

    int* getData() { return data.get(); }
    size_t size() const { return strings.size(); }
};

// Usage demonstrates automatic resource management
ModernResourceManager createManager() {
    return ModernResourceManager(100);  // ✅ Move, not copy
}
```

# What You Learned

## Key Takeaways

- ✓ Encapsulation, Inheritance, Polymorphism
- ✓ The Rule of Three/Five/Zero
- ✓ Common Pitfalls
- ✓ Virtual Functions in Constructors and Destructors

*→ Swipe to learn →*